PYTHON FUNCTIONS

Fundamentals and Advanced Function Concepts

Purpose A student-friendly guide to understanding how Python functions are defined, how information is passed between functions, and how functions can be combined to build reusable programs.

How to use this document

Read the explanation for each concept, study the example, and run the code in Python. Try changing the input values to see how the output changes. The examples are designed to be small enough to understand without needing a large project.

Prerequisites

- Variables, data types and operators
- Conditional statements and loops
- Basic lists and dictionaries
- Basic function definition and function calls

Contents

Part	Topics
Part 1 – Function Fundamentals	Defining and calling functions; positional and keyword arguments; default arguments; variable scope; *args and **kwargs; lambda functions; map(), filter() and reduce(); docstrings; modules and standard modules.
Part 2 – Advanced Function Concepts	Nested functions; functions as arguments; returning functions; closures; decorators; recursive functions; function annotations; positional-only and keyword-only arguments.
At the end	How the concepts connect; quick reference; practice exercises; revision questions.

Introduction: What is a function?

A function is a named, reusable block of code that performs a particular task. Instead of writing the same instructions repeatedly, we can place them in a function and call that function whenever needed.

```
def greet():
    print("Hello! Welcome to Python.")

greet()
greet()
```

Expected output:

```
Hello! Welcome to Python.
Hello! Welcome to Python.
```

Think of a function A function is like a small machine: it can receive information (inputs), perform a task, and optionally send a result back (return value).

Part 1 – Function Fundamentals

These are the essential concepts used to define, call and organise functions in Python.

1. Defining and calling functions

What it means: Use the def keyword to define a function. The indented statements form its body. The function runs when it is called by writing its name followed by parentheses.

```
def show_message():
    print("Learning Python functions")

show_message()
```

Expected output: `Learning Python functions`

How to understand it: The line beginning with def creates the function. The call show_message() runs the code inside it. Defining a function does not execute its body.

Why it is useful: Functions help divide a program into manageable parts, reduce repeated code and make programs easier to test and maintain.

2. Positional and keyword arguments

What it means: Arguments are values supplied when a function is called. With positional arguments, values are matched to parameters by their position. With keyword arguments, values are matched by parameter name.

```
def describe_student(name, age):
    print("Name:", name)
    print("Age:", age)

describe_student("Meera", 20)          # Positional
describe_student(age=20, name="Meera") # Keyword
```

Expected output: `Both calls display Meera and 20.`

How to understand it: In the first call, 'Meera' goes to name because it is first, and 20 goes to age. In the second call, the parameter names make the matching explicit, so the order can differ.

Why it is useful: Keyword arguments can make calls easier to read, especially when a function has several parameters.

3. Default arguments

What it means: A parameter can be given a default value. If the caller does not supply that argument, Python uses the default.

```
def greet(name, message="Good morning"):
    print(message, name)

greet("Asha")
greet("Asha", "Welcome")
```

Expected output: `Good morning Asha`
`Welcome Asha`

How to understand it: In the first call, message is omitted, so the default 'Good morning' is used. In the second call, 'Welcome' replaces the default.

Why it is useful: Default arguments make functions flexible while keeping simple calls short.

4. Variable scope: local and global

What it means: Scope describes where a variable can be accessed. A local variable is created inside a function and is normally available only there. A global variable is defined outside functions and can be read inside them.

```
school = "Green Valley"  # Global variable

def show_school():
    student_count = 30       # Local variable
    print(school)
    print(student_count)

show_school()
print(school)
# print(student_count)  # This would cause a NameError
```

Expected output: `Green Valley`
`30`
`Green Valley`

How to understand it: student_count belongs to the function call and is not available outside it. school is defined at module level, so the function can read it.

Why it is useful: Understanding scope helps avoid naming conflicts and makes it clear where data is created and used.

Remember Use global variables sparingly. Prefer passing values as arguments and returning results where practical.

5. *args and **kwargs

What it means: *args allows a function to receive any number of additional positional arguments. **kwargs allows it to receive any number of additional keyword arguments. The names args and kwargs are conventions; the * and ** symbols provide the behaviour.

```
def add_numbers(*args):
    total = 0
    for number in args:
        total += number
    print("Total:", total)

def show_details(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)

add_numbers(10, 20, 30)
show_details(name="Riya", age=21)
```

Expected output: `Total: 60`
`name : Riya`
`age : 21`

How to understand it: Inside add_numbers, args behaves like a tuple containing the extra positional values. Inside show_details, kwargs behaves like a dictionary containing the keyword names and values.

Why it is useful: These features are useful when the number or names of supplied arguments may vary.

6. Lambda functions

What it means: A lambda is a small anonymous function written as a single expression. It can accept arguments and returns the value of its expression automatically.

```
square = lambda number: number * number

print(square(5))
print((lambda a, b: a + b)(10, 20))
```

Expected output: `25`
`30`

How to understand it: The first lambda takes number and calculates its square. The second takes a and b and returns their sum. A lambda does not use a return statement.

Why it is useful: Lambdas are useful for short functions, especially when a function is needed briefly as an argument to another operation.

Remember Use a normal def function when the logic needs multiple statements or would be clearer with a descriptive function name.

7. map(), filter() and reduce()

What it means: These tools apply functions to collections or combine their values. map() transforms each item; filter() keeps items that meet a condition; reduce() combines items into one accumulated result.

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda n: n * n, numbers))
evens = list(filter(lambda n: n % 2 == 0, numbers))
total = reduce(lambda a, b: a + b, numbers)

print(squares)
print(evens)
```

```
print(total)
```

Expected output: `[1, 4, 9, 16, 25]`
`[2, 4]`
`15`

How to understand it: map applies the square operation to every number. filter tests each number and keeps the even ones. reduce adds values cumulatively: (((1+2)+3)+4)+5.

Why it is useful: They provide compact ways to transform, select and combine data. In Python, list comprehensions are often a readable alternative to map() and filter().

8. Docstrings

What it means: A docstring is a string written as the first statement inside a function, class or module. It describes what that piece of code does.

```
def calculate_area(length, width):
    """Return the area of a rectangle."""
    return length * width

print(calculate_area(5, 3))
print(calculate_area.__doc__)
```

Expected output: `15`
`Return the area of a rectangle.`

How to understand it: The triple-quoted text is the function's docstring. Python makes it available through the __doc__ attribute, and documentation tools can use it.

Why it is useful: Docstrings help other programmers (and your future self) understand the purpose, inputs and behaviour of code.

9. Modules and standard modules

What it means: A module is a Python file containing code such as functions, variables and classes that can be reused by importing it. Python also provides a standard library of modules for common tasks.

```
# Example using a standard-library module
import math

print(math.sqrt(49))
print(math.pi)

# You can also import a specific function:
from math import ceil
print(ceil(4.2))
```

Expected output: `7.0`
`3.141592653589793`
`5`

How to understand it: import math makes the math module available, so its functions and values are accessed using math.name. from math import ceil imports that function directly.

Why it is useful: Modules help organise larger programs and let you reuse existing Python functionality.

Part 2 – Advanced Function Concepts

These concepts build on the fundamentals. They show how functions can be placed inside other functions, passed around like values, returned, and extended with reusable behaviour.

1. Nested functions

What it means: A nested function is a function defined inside another function. The inner function is defined when the outer function runs and is usually called from within the outer function.

```
def outer():
    print("Outer function is running")

    def inner():
        print("Inner function is running")

    inner()

outer()
```

Expected output: `Outer function is running`
`Inner function is running`

How to understand it: inner is defined inside outer. The call to inner() is made from within outer.

Why it is useful: Nested functions are useful for helper operations that belong to one function and are not needed elsewhere.

2. Functions as arguments

What it means: In Python, a function is an object. This means it can be assigned to a variable, passed as an argument to another function, and called through that variable.

```
def square(number):
    return number * number

def doublenum (number): # another func
    return number * 2

def apply_operation(operation, value):
    result = operation(value)
    print("Result:", result)

apply_operation(square, 5)
apply_operation(doublenum, 5)
```

Expected output: Result: 25 Result: 10

How to understand it: The name square (without parentheses) refers to the function itself. When square is passed to apply_operation(), the parameter operation refers to the square function. Therefore, operation(value) is equivalent to calling square(5), which returns 25. Similarly, when doublenum is passed, operation(value) is equivalent to calling doublenum(5), which returns 10.

Why it is useful: This allows one function to provide a general process while another function supplies the specific operation.

3. Returning functions

What it means: A function can return another function, just as it can return a number or a string. Notice the difference between returning a function name and calling that function.

```
def make_tax_calculator(rate):
    def calculate(price):
        return price + (price * rate / 100)
    return calculate          # no brackets: returning the function itself

gst_18 = make_tax_calculator(18)
gst_5  = make_tax_calculator(5)

print(gst_18(1000))   # 1180.0
print(gst_5(1000))    # 1050.0
```

Expected output: 1180.0

How to understand it: make_tax_calculator(18) does not calculate any tax. It builds and returns a new function, calculate, It manufactures a tax calculator which already has the rate 18 built in. That function is stored in gst_18. Calling gst_18(1000) then runs calculate with price = 1000, using the remembered rate. gst_5 is a completely separate function with its own remembered rate = 5.
Same blueprint, two independent products - make_tax_calculator is like a function factory

What happens, step by step

1. make_tax_calculator(18) runs. Inside, it defines calculate and then executes return calculate.
2. There are no brackets after calculate. That's the whole point. return calculate hands back the function object itself; return calculate(...) would run it and hand back a number.
3. The outer function finishes, but the inner function survives, carrying rate = 18 with it. This is a closure: the inner function remembers the variables from the scope where it was created, even after that scope has ended.
4. gst_18 now holds that function. Calling gst_18(1000) runs calculate with rate already baked in as 18.

Why it is useful: You configure the behaviour once (the tax rate) and reuse the resulting function many times without passing the rate again. It also keeps related settings private to each function, and it is the foundation for decorators and callbacks.

Remember: return calculate (no brackets) returns the function itself, while return calculate(100) runs it and returns the answer. The inner function keeps the outer function's values (here, rate) even after the outer function has finished. That is what makes it a closure.

4. Closures

What it means: A closure occurs when an inner function retains access to variables from its enclosing function, even after the enclosing function has finished running.

```
def make_multiplier(factor):
    def multiply(number):
        return number * factor

    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))
```

```
print(triple(5))
```

Expected output: `10`
`15`

How to understand it: make_multiplier creates and returns multiply. The returned functions retain the factor value from the call that created them: 2 for double and 3 for triple.

Why it is useful: Closures are useful for creating specialized functions that remember settings or state without using global variables.

5. Decorators

What it means: A decorator is a function that takes another function and returns a function that adds behaviour around it. The @ syntax is a convenient way to apply a decorator.

```
def announce(func):
    def wrapper():
        print("Starting the task")
        func()
        print("Task finished")
    return wrapper

@announce
def do_task():
    print("Doing the task")

do_task()
```

Expected output: `Starting the task`
`Doing the task`
`Task finished`

How to understand it: The decorator receives do_task as func. It creates wrapper, which prints a message, calls the original function, and prints another message. The decorator returns wrapper. Writing @announce is equivalent to do_task = announce(do_task).

Why it is useful: Decorators are used for reusable features such as logging, timing, authentication checks and caching.

A practical decorator note This first example works with a function that takes no arguments. A more flexible decorator uses *args and **kwargs so the wrapper can pass through different arguments.

```
def announce(func):
    def wrapper(*args, **kwargs):
        print("Starting")
        result = func(*args, **kwargs)
        print("Finished")
        return result
    return wrapper

@announce
def greet(name):
    print("Hello", name)

greet("Asha")
```

The wrapper forwards positional and keyword arguments to the original function. It also returns the original function's result, so that result is not lost.

6. Recursive functions

What it means: A recursive function calls itself to solve a smaller version of the same problem. It must have a base case that stops the calls; otherwise, it may continue until Python raises a recursion error.

```
def factorial(number):
    if number == 1:       # Base case
        return 1

    return number * factorial(number - 1)

print(factorial(5))
```

Expected output: `120`

How to understand it: factorial(5) becomes 5 * factorial(4), then 4 * factorial(3), and so on until factorial(1) returns 1. The pending calls then finish their calculations.

Why it is useful: Recursion can be a natural way to solve problems involving nested structures, trees and divide-and-conquer algorithms.

7. Positional-only and keyword-only arguments

Python allows a function to control how its parameters must be supplied when the function is called

Program 1 – Positional-only arguments

Concept: Parameters defined before the / symbol must be passed by position. They cannot be supplied using parameter names.

```
def calculate(a, b, /):
    print(a + b)

calculate(10, 20)       # Valid
# calculate(a=10, b=20)  # Error
```

Expected output:

`30`

How to understand it: The / symbol indicates that a and b are positional-only parameters. In the function call, Python assigns 10 to a and 20 to b based on their positions. If the parameters are supplied using names, as shown in the commented line, Python raises a TypeError.

Program 2 – Keyword-only arguments

Concept: Parameters defined after the * symbol must be passed using their parameter names. They cannot be supplied by position.

```
def display(name, *, age):
    print(name, age)

display("John", age=25)  # Valid
# display("John", 25)    # Error
```

Expected output:

```
John 25
```

How to understand it: The * symbol indicates that parameters after it are keyword-only. In this example, name can be supplied by position, but age must be supplied using its parameter name. Passing 25 by position, as shown in the commented line, raises a TypeError.

Program 3 – Combining positional-only and keyword-only arguments

Concept: A function can use both / and * to define positional-only and keyword-only parameters. Parameters between these symbols can be supplied either by position or by keyword.

```
def describe(name, /, age, *, city):
    print(name, age, city)

describe("Arun", 22, city="Bengaluru")
describe("Meera", age=21, city="Mysuru")
```

Expected output:

```
Arun 22 Bengaluru
Meera 21 Mysuru
```

How to understand it: name must be supplied by position because it appears before /. age can be supplied positionally or by keyword because it appears between / and *. city must be supplied by keyword because it appears after *.

Quick reference

- / – Parameters before it are positional-only.
- * – Parameters after it are keyword-only.

8. Function annotations

What it means: Annotations let you attach information about the expected types of parameters and the return value. They improve documentation and help tools analyze code, but Python does not automatically enforce them.

```
def add(a: int, b: int) -> int:
    return a + b

print(add(10, 20))
print(add.__annotations__)
```

Expected output: `30`

```
{'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
```

How to understand it: a: int and b: int annotate the parameters. -> int annotates the expected return type. These are hints, not runtime restrictions: Python may still allow other types if the operation supports them.

Why it is useful: Annotations make function interfaces clearer and can help editors and static type-checking tools.

How the concepts connect

Some function concepts are closely related. Understanding the connections makes the later topics easier.

Concept group	Topics	Connection
Basic function use	Defining/calling, parameters, arguments, defaults, scope	The foundation for writing reusable functions.
Flexible function inputs	*args, **kwargs, positional-only and keyword-only arguments	Different ways to accept and control arguments.
Functions as values	Functions as arguments; returning functions	Functions can be passed around and returned like other objects.
Functions inside functions	Nested functions; closures	Inner functions can use names from enclosing scopes.
Function enhancement	Decorators	Uses functions as arguments, nested functions and returning functions to wrap behaviour.
Other function techniques	Recursion; annotations; lambda	Techniques and metadata that support particular programming needs.
Functional tools	map(), filter(), reduce()	Often use functions (including lambdas) as inputs.

Quick reference

Term / syntax	What to remember
def	Defines a function.
return	Ends a function call and sends a result back.
yield	Produces a value from a generator and pauses its execution.
*args	Collects extra positional arguments into a tuple.
**kwargs	Collects extra keyword arguments into a dictionary.
lambda	Creates a small anonymous function expression.
map()	Applies a function to each item.
filter()	Keeps items for which a condition is true.
reduce()	Combines items into one accumulated result.
__doc__	Provides access to a function's docstring.
/	Marks preceding parameters as positional-only.
*	When used alone in a parameter list, marks following parameters as keyword-only.

Student practice exercises

Try these after studying the relevant sections. Write and run each program yourself before checking with a trainer.

1. Function fundamentals – Write a function called calculate_total(price, quantity) that returns price multiplied by quantity. Call it with positional arguments and then with keyword arguments.

2. Default argument – Write a function greet(name, message='Welcome') that prints the message and name. Call it once using the default and once with a different message.

3. *args – Write a function that accepts any number of numbers and returns their sum. Test it with two, four and no arguments.

4. **kwargs – Write a function that accepts keyword details about a product and prints each key and value.

5. Lambda and map – Given a list of prices, use map() and a lambda to calculate prices after adding 10% tax. Convert the result to a list to display it.

6. filter() and reduce() – From a list of numbers, use filter() to keep numbers greater than 10. Then use reduce() to calculate their total.

7. Nested function – Create an outer function that defines and calls an inner function to display a message.

8. Function as argument – Write a function apply_twice(operation, value) that applies the supplied function twice. Test it with a function that adds 3 to a number.

9. Returning a function – Write make_power(exponent) that returns an inner function. Use it to create square and cube functions.

10. Closure – Extend make_power so the returned function remembers the exponent supplied when it was created.

11. Decorator – Create a decorator that prints 'Before calling' and 'After calling' around a function. Then adapt it to support a function with an argument.

12. Recursion – Write a recursive function that calculates the sum of integers from 1 to n. Identify its base case.

13. Annotations – Add parameter and return annotations to a function that converts Celsius to Fahrenheit. Check its __annotations__ attribute.

14. Argument restrictions – Define a function with one positional-only parameter and one keyword-only parameter. Try valid and invalid calls and observe the errors.

Revision questions

1. What is the difference between a parameter and an argument?
2. What is the difference between a positional argument and a keyword argument?
3. When does Python use a default argument value?
4. What is the difference between a local and a global variable?
5. What do *args and **kwargs collect?
6. What is the difference between a lambda function and a function defined with def?
7. How do map(), filter() and reduce() differ?
8. What is a docstring, and how can you access it?
9. What is a nested function?
10. What does it mean to pass a function as an argument?
11. What is the difference between return inner and return inner()?
12. What does a closure remember?
13. What is the purpose of a decorator?
14. Why does a recursive function need a base case?
15. Do function annotations enforce types at runtime?
16. What do / and * indicate in a function parameter list?